



The Apache Spark Big Data Analytics Tool: Fast and Interactive

Apache Spark is an open source data analytics cluster computing framework that is extremely fast. It is versatile and can be used in tandem with other applications like Hadoop. Spark became an Apache Top Level Project in February 2014.

We have already witnessed the power of the Hadoop framework in Big Data analytics. It has proven its enormous capability to solve the most critical data-intensive challenges. While the Hadoop framework is perfect for the batch and stored kind of Big Data job processing, on its own it proves inadequate when it comes to real time and interactive Big Data analytics.

In this article, we are going to explore the power of Apache Spark, which has been recently developed by researchers from the open source community. It works on in-memory data concepts that make it a 100 times faster than the Hadoop MapReduce framework. It works very effectively for querying very large data sets and also provides sub-second latency of processed data sets.

What is interactive Big Data analytics?

When users need to perform interactive ad-hoc queries based on machine learning, graph processing algorithms and fast processing in real time—all involving massive volumes of data—it is known as interactive Big Data analytics.

After the advent of the Hadoop MapReduce framework, this is a very widespread area of research. Nowadays, more research is focused on highly interactive and fast Big Data processing.

Existing iterative, interactive and streaming tools

Here are some iterative, interactive and streaming tools of Big Data.

Dremel: A Google product launched in 2010, it works on a novel approach. It offers a highly interactive, real-time and ad-hoc query interface, which is not possible with MapReduce.

ClouderaImpala: This is an SQL query engine that gets executed in Apache Hadoop, and is a leading enterprise product with scalable parallel database technology and the strength of Hadoop. It allows users to directly raise queries stored in HDFS (Hadoop distributed file system) and Apache HBase (no data transfer is needed).

The Apache Spark approach

The first approach to Apache Spark is in-memory computing,

which means data gets stored or cached in the distributed main memory instead of the disk.

In-memory computing: Apache Spark introduces a new data primitive called RDDs (resilient distributed datasets), with the help of which data is stored across the cluster's memory. When data is in-memory, there is no need for the replication of data sets. This approach automatically provides fault tolerance capabilities. This RDDS provides a read and write capability that is 40 times faster than HDFS or any other distributed file system.

Hadoop interoperability: Apache Spark is also fully interoperable with Hadoop and is, therefore, being adopted by many research projects. Read and write can be performed easily by any existing storage system that is supported by Hadoop like HDFS, HBase, S3 (simple storage system) and even with the input/output APIs of Hadoop. Therefore, Apache Spark can be used very effectively for non-interactive applications as well. Apart from this, Spark can also be integrated with Mlib (a machine learning tool), Spark streaming, Shark and Graphx.

Advantages of Apache Spark

The following are the key advantages of Apache Spark.

- Fastest data processing:** Initially, the main purpose of deploying Spark was to enhance efficiency in the current MapReduce applications. Actually, MapReduce is a general framework and is not specifically implemented in core Hadoop. Spark also enables MapReduce, as it can use memory optimally (even in recovering from failure cases). Some features work faster in Spark's MapReduce compared to Hadoop's MapReduce, even without making efficient use of caches in iterations.
- The iterative algorithm:** By using the `cache()` function, Spark provides users and applications the facility to provide cached datasets in explicit form. It means that all the applications can take data from the RAM rather than the disk. This dramatically increases the performance of the iterative algorithm, which reaches the same dataset again and again.